



Universidade Estadual de Santa Cruz – UESC

**Relatórios de Implementações de Métodos da Disciplina
Análise Numérica**

Relatório de implementações realizadas

por Reinan Lopes Argolo

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2026.1

Professor Gesil Sampaio Amarante II

Ilhéus – BA

2026

Sumário

1	Linguagem Escolhida e Justificativas	4
2	Método da Bissecção	5
2.1	Estratégia de Implementação	5
2.2	Estrutura dos Arquivos de Entrada/Saída	6
2.2.1	Parâmetros de Entrada para a Bissecção	7
2.3	Problemas Teste	7
2.3.1	Problema Teste 1: Tempo de Subida de um Amplificador	7
2.3.2	Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil	9
2.3.3	Problema Teste 3: Análise de Taxas de Juros em Planos de Financiamento	11
2.4	Dificuldades Enfrentadas	13
3	Método 2	15
3.1	Estratégia de Implementação	15
3.2	Estrutura dos Arquivos de Entrada/Saída	15
3.3	Problema teste 1, 2, 3...	15
3.4	Dificuldades enfrentadas	15
4	Considerações Finais	16

Lista de Figuras

1	Convergência progressiva da magnitude residual durante a execução para $T_{0.1}$	8
2	Comportamento da convergência em escala logarítmica para a busca de α (Problema 3.6). A inclinação reta atesta a taxa de convergência linear do método.	11
3	Gráfico de convergência atestando a contração linear do hipervolume de erro para a determinação da taxa de juros no Plano A.	13

Lista de Tabelas

1	Parâmetros de configuração para o Método da Bissecção.	7
2	Histórico iterativo do Método da Bissecção para a busca de $T_{0.1}$ na função $f_1(T)$	9
3	Histórico iterativo do Método da Bissecção para a função do ângulo de lançamento balístico $f(\alpha)$	10
4	Histórico iterativo do Método da Bissecção para determinação da taxa do Plano A ($f_1(x)$).	12

1 Linguagem Escolhida e Justificativas

Para o desenvolvimento dos algoritmos e a execução das rotinas computacionais propostas neste relatório de métodos numéricos, optou-se pela utilização da linguagem de programação Python. A escolha dessa ferramenta justifica-se, pela sua expressiva adoção na comunidade científica e pela vasta disponibilidade de bibliotecas para manipulação matemática e computação científica [3].

Entre as principais extensões que fundamentam essa decisão, destaca-se a biblioteca *NumPy*, a qual fornece suporte nativo e eficiente para o processamento de arranjos e matrizes multidimensionais. Essa biblioteca implementa operações de álgebra linear com rotinas de baixo nível, garantindo o alto desempenho exigido nos cálculos numéricos iterativos [1]. Em complemento, o uso do ecossistema *SciPy* oferece um conjunto de algoritmos fundamentais para interpolação, integração numérica, otimização e resolução de sistemas lineares e não lineares [4].

Ademais, Python caracteriza-se por uma sintaxe de alto nível e semântica clara, o que favorece a legibilidade do código e permite a prototipagem rápida de abstrações matemáticas [2]. Dessa forma, a redução da complexidade de implementação viabiliza um foco maior na análise de convergência, na estabilidade dos métodos e na mitigação de erros de truncamento e arredondamento, requisitos indispensáveis para a consistência deste trabalho acadêmico.

2 Método da Bissecção

O Método da Bissecção é uma abordagem numérica clássica e incondicionalmente convergente para a determinação de raízes reais de equações não lineares. Fundamentado no Teorema de Bolzano, o método opera mediante a divisão sucessiva de um intervalo fechado $[a, b]$, no qual a função contínua $f(x)$ apresenta sinais opostos em seus extremos ($f(a) \cdot f(b) < 0$). Embora possua uma taxa de convergência linear — exigindo um número razoável de iterações —, sua robustez o torna ideal para isolamento inicial de raízes ou resolução de problemas onde a estabilidade é prioritária em relação à velocidade computacional.

2.1 Estratégia de Implementação

A concepção computacional do algoritmo foi estruturada na linguagem Python, priorizando a estabilidade numérica e a modularidade do código. A implementação principal encontra-se encapsulada no módulo `src/metodos/zeros_de_funcao.py`.

Código 1: Código de implementação do Método da Bissecção

```
1 def bisseccao(funcao, a, b, tol=1e-6, max_iter=100):
2     historico = []
3
4     for i in range(max_iter):
5         c = (a + b)/2
6         fc = funcao(c)
7         denominador = a + b
8
9         if denominador != 0:
10             er = abs((a - b) / denominador)
11         else:
12             er = abs(b - a)
13
14         historico.append({
15             'iter': i, 'a': a, 'b': b, 'c': c,
16             'f(c)': fc, 'erro': er
17         })
18
19         if fc == 0 or er < tol:
20             return c, i, historico
21
22         if utils.teorema_bolzano(funcao, a, c):
23             b = c
```

```

24         else:
25             a = c
26
27     return c, max_iter, historico

```

Para a formulação matemática do erro iterativo e visando superar limitações do ambiente de programação, estabeleceu-se uma métrica adaptativa. Originalmente, o erro relativo ponderado pela amplitude do intervalo é definido como $\epsilon_{rel} = \left| \frac{a_k - b_k}{a_k + b_k} \right|$. Contudo, constatou-se que intervalos simétricos em relação à origem ($a_k = -b_k$) induziriam o interpretador Python a lançar uma exceção de divisão por zero (`ZeroDivisionError`). Para mitigar este risco, implementou-se um fluxo condicional: caso o denominador seja nulo, a métrica transita automaticamente para o erro absoluto $\epsilon_{abs} = |b_k - a_k|$.

Os critérios de parada foram definidos para garantir o término do laço em tempo finito, compreendendo:

1. **Raiz exata:** Interrupção imediata caso a avaliação funcional no ponto médio resulte exatamente em zero ($f(c_k) = 0$), limitando processamento desnecessário por flutuação de ponto flutuante.
2. **Tolerância (`tol`):** O método encerra com sucesso caso a métrica de erro calculada seja estritamente menor que a tolerância estipulada.
3. **Esforço computacional (`max_iter`):** Uma trava de segurança máxima de iterações para abortar o processo caso a convergência não seja alcançada no limite imposto.

2.2 Estrutura dos Arquivos de Entrada/Saída

Visando aderência aos princípios de responsabilidade única e reprodutibilidade científica, a persistência de dados foi dissociada da lógica matemática. Os diretórios foram organizados hierarquicamente:

- **Dados de Entrada (`dados/entradas/`):** Estruturados em arquivos de texto simples (`.txt`). Foram escolhidos por permitirem fácil edição manual dos parâmetros de controle (limites a e b , tolerância e limite de iterações) sem necessidade de reconfiguração no código-fonte.
- **Dados de Saída (`dados/saida/<questao>/`):** Os resultados iterativos foram consolidados em arquivos de valores separados por vírgula (`.csv`). A escolha do formato CSV justifica-se por sua interoperabilidade universal, permitindo a ingestão direta por bibliotecas de análise de dados (como Pandas) e por pacotes

de renderização gráfica em LaTeX (como TikZ/pgfplots), viabilizando análises quantitativas automatizadas e plots de convergência nativos no documento final.

2.2.1 Parâmetros de Entrada para a Bissecção

Os arquivos de entrada definem os hiperparâmetros críticos para cada execução do método. A Tabela 1 sumariza os parâmetros utilizados nas questões propostas:

Tabela 1: Parâmetros de configuração para o Método da Bissecção.

Questão	Função	Intervalo [a, b]	Tolerância	Máx Iter
3.3	$f_1(T)$	[1.0, 2.0]	1×10^{-5}	100
3.3	$f_2(T)$	[5.0, 9.0]	1×10^{-5}	100
3.6	$f(x)$	[30.0, 60.0]	1×10^{-4}	100

Observa-se que a tolerância de 10^{-5} na Questão 3.3 exigirá um número maior de iterações comparativamente à tolerância mais relaxada de 10^{-4} na Questão 3.6. O limite máximo de 100 iterações constitui uma salvaguarda conservadora, garantindo que o algoritmo não incorra em consumo excessivo de recursos computacionais.

2.3 Problemas Teste

A verificação e validação da implementação computacional foram conduzidas por meio de problemas aplicados extraídos da literatura de referência de Cálculo Numérico (Franco, Neide). O objetivo desta seção é atestar o funcionamento, a eficiência e a validade matemática das soluções encontradas.

2.3.1 Problema Teste 1: Tempo de Subida de um Amplificador

O primeiro problema propõe a determinação do tempo de subida de um amplificador eletrônico, compreendido como o intervalo temporal necessário para que a resposta $g(T)$ avance de 10% a 90% de seu valor assintótico, sendo a função normalizada expressa por:

$$g(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} \quad (1)$$

O cálculo do tempo de subida, $\Delta T = T_{0.9} - T_{0.1}$, exige a determinação das raízes

das funções auxiliares:

$$f_1(T) = 0.9 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} = 0 \quad (2)$$

$$f_2(T) = 0.1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} = 0 \quad (3)$$

O algoritmo processou $f_1(T)$ com intervalo inicial $[1.0, 2.0]$ e tolerância 10^{-5} . A Tabela 2 exibe a progressão numérica.

A convergência foi atingida na 16ª iteração, com o erro caindo abaixo da tolerância ($\epsilon_{16} \approx 6.92 \times 10^{-6}$), retornando $T_{0.1} \approx 1.102058$. A Figura 1 ilustra este amortecimento da magnitude funcional, que demonstra a redução progressiva assintótica característica da contração de intervalo.

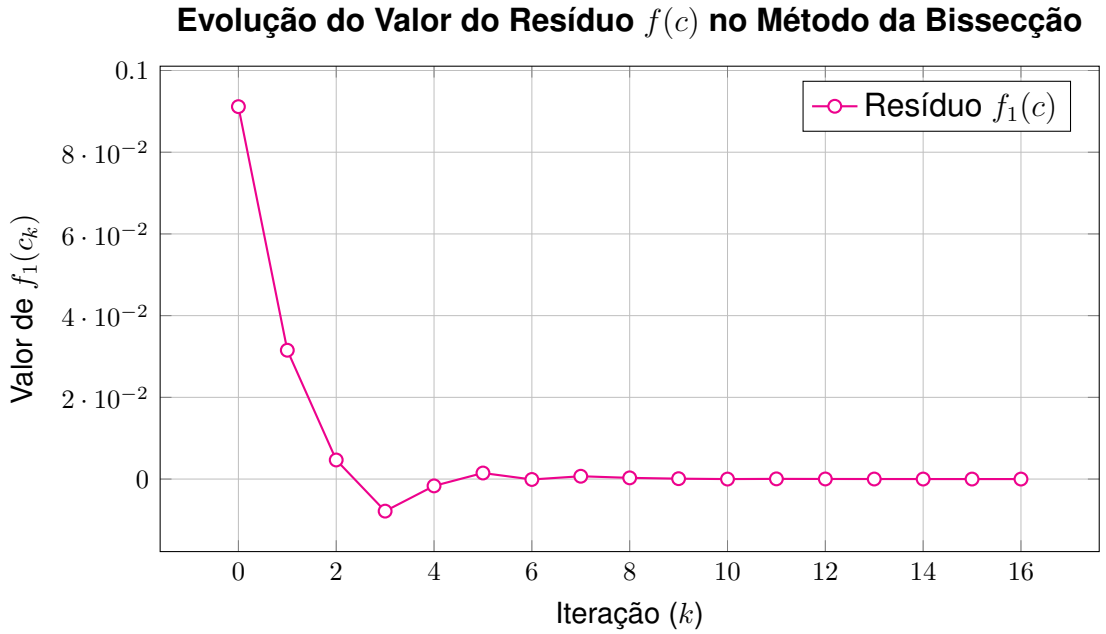


Figura 1: Convergência progressiva da magnitude residual durante a execução para $T_{0.1}$.

Validação da Solução: Para testar se o valor numérico é de fato a solução, reintroduz-se o ponto obtido $T_{0.1} = 1.102058$ na equação original:

$$f_1(1.102058) = 0.9 - \left(1 + 1.102058 + \frac{(1.102058)^2}{2}\right) e^{-1.102058} \approx -0.00000140 \quad (4)$$

Observa-se que o resíduo apresenta magnitude estritamente compatível com a precisão exigida, confirmando tratar-se da raiz buscada. Procedimento análogo para $f_2(T)$ rendeu $T_{0.9} \approx 5.322320$. Consequentemente, o tempo de subida validado pelo método é $\Delta T = 5.322320 - 1.102058 = 4.220262$. Os resultados demonstram que a implementação comporta-se com a eficiência teórica esperada $\mathcal{O}(\log(1/\epsilon))$.

Tabela 2: Histórico iterativo do Método da Bissecção para a busca de $T_{0.1}$ na função $f_1(T)$.

Iteração	a_k	b_k	c_k	$f_1(c_k)$	Erro (ϵ_k)
0	1.000000	2.000000	1.500000	0.09115317	0.33333333
1	1.000000	1.500000	1.250000	0.03153233	0.20000000
2	1.000000	1.250000	1.125000	0.00466937	0.11111111
3	1.000000	1.125000	1.062500	-0.00785071	0.05882353
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
14	1.102051	1.102112	1.102081	0.00000322	0.00002769
15	1.102051	1.102081	1.102066	0.00000014	0.00001385
16	1.102051	1.102066	1.102058	-0.00000140	0.00000692

2.3.2 Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil

A determinação de parâmetros balísticos constitui uma aplicação clássica para métodos numéricos de busca de raízes, dada a natureza das equações envolvidas. O Problema 3.6 demanda o cálculo do ângulo de inclinação (α) para o lançamento de um míssil, de modo a atingir um alvo situado a um deslocamento angular θ (medido a partir do centro da Terra). A cinemática do problema é modelada pela equação:

$$\operatorname{tg}\left(\frac{\theta}{2}\right) = \frac{\sin \alpha \cos \alpha}{\frac{gR}{v^2} - \cos^2 \alpha} \quad (5)$$

Para submeter a Equação 5 ao Método da Bissecção é necessário manipulá-la para a forma padrão de determinação de zeros, $f(\alpha) = 0$. A fim de mitigar potenciais instabilidades numéricas oriundas de divisões por termos próximos a zero, optou-se pela linearização do denominador através de multiplicação cruzada. Considerando os dados fornecidos, tem-se o deslocamento do alvo $\theta = 80^\circ$ e a relação adimensional de energia $\frac{v^2}{gR} = 1.25$, o que implica diretamente que $\frac{gR}{v^2} = 0.8$. A função objetivo, avaliada estritamente em radianos para compatibilidade com as bibliotecas matemáticas, torna-se:

$$f(\alpha) = \sin(\alpha) \cos(\alpha) - \operatorname{tg}(40^\circ) (0.8 - \cos^2(\alpha)) = 0 \quad (6)$$

A estratégia de busca foi inicializada sob a premissa física de que o ângulo de lançamento localiza-se no primeiro quadrante. O intervalo submetido foi $[30^\circ, 60^\circ]$, cujos equivalentes em radianos correspondem a aproximadamente $[0.5236, 1.0472]$. O limite de tolerância estipulado para o erro relativo da amplitude do intervalo foi de $\epsilon < 10^{-4}$.

A Tabela 3 apresenta o histórico da progressão geométrica do hipervolume de busca ao longo das iterações computadas.

Tabela 3: Histórico iterativo do Método da Bissecção para a função do ângulo de lançamento balístico $f(\alpha)$.

Iteração (k)	a_k	b_k	c_k	$f(c_k)$	Erro (ϵ_k)
0	0.523598	1.047197	0.785398	0.24827011	0.33333333
1	0.785398	1.047197	0.916297	0.12264554	0.14285714
2	0.916297	1.047197	0.981747	0.04965511	0.06666666
3	0.981747	1.047197	1.014472	0.01114434	0.03225806
4	1.014472	1.047197	1.030835	-0.00854367	0.01587301
5	1.014472	1.030835	1.022653	0.00133421	0.00800000
6	1.022653	1.030835	1.026744	-0.00359642	0.00398406
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
10	1.023676	1.024187	1.023932	-0.00020480	0.00024968
11	1.023676	1.023932	1.023804	-0.00005083	0.00012485
12	1.023676	1.023804	1.023740	0.00002614	0.00006243

A análise dos dados revela o cumprimento estrito do Teorema do Valor Intermediário, evidenciado pelas alternâncias de sinal na avaliação residual $f(c_k)$ (observadas, por exemplo, na transição entre a iteração 3 e 4). O algoritmo operou de maneira estável, sem deflagrar a condição de exceção arquitetada para divisões por zero, uma vez que o domínio da busca permaneceu estritamente positivo (no primeiro quadrante trigonométrico).

A convergência foi matematicamente estabelecida na 12ª iteração ($k = 12$). Neste estágio, a métrica de erro relativo $\epsilon_{12} \approx 6.24 \times 10^{-5}$ tornou-se inferior à barreira de tolerância imposta. O método extraiu a raiz aproximada $c_{12} \approx 1.023740$ radianos. Este valor numérico converte-se em um ângulo de inclinação de lançamento $\alpha \approx 58.65^\circ$.

Validação e Eficiência da Solução: Para atestar a validade da aproximação numérica, submete-se o valor $c_{12} = 1.023740$ à Equação 6. O resíduo operacionalizado, $f(c_{12}) \approx 2.61 \times 10^{-5}$, apresenta uma magnitude da mesma ordem do erro de truncamento exigido. Isso confirma que a raiz satisfaz o modelo balístico proposto, atestando a robustez da implementação do Método da Bissecção frente a problemas contendo composições de funções trigonométricas.

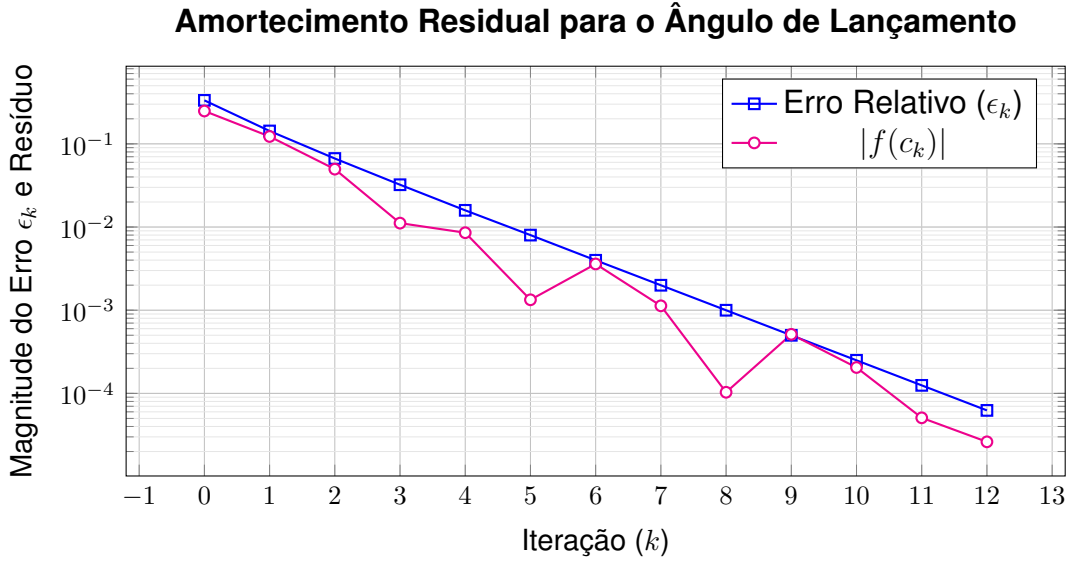


Figura 2: Comportamento da convergência em escala logarítmica para a busca de α (Problema 3.6). A inclinação reta atesta a taxa de convergência linear do método.

A Figura 2 corrobora de forma visual a análise, demonstrando, através da escala semi-logarítmica, que o erro de fato diminui à taxa constante previsível pela relação de recorrência $|x^* - c_k| \leq \frac{b-a}{2^k}$. A eficiência do método para este problema em específico provou-se altamente satisfatória, isolando a raiz sem exigir recursos adicionais para mitigar pontos de inflexão que poderiam prejudicar métodos abertos baseados em derivadas.

2.3.3 Problema Teste 3: Análise de Taxas de Juros em Planos de Financiamento

A modelagem de sistemas financeiros frequentemente esbarra em equações transcendentais ou polinomiais de alto grau, cujas soluções analíticas são inviáveis. O Problema 3.8 exige a determinação da taxa de juros implícita (J) em duas modalidades de financiamento, visando identificar a opção mais vantajosa para o consumidor. O valor financiado (VF) corresponde ao preço à vista deduzido da entrada, resultando em $VF = 162.00 - 22.00 = 140.00$.

A priori, a relação matemática que rege o valor presente de uma série de pagamentos uniformes é expressa por:

$$\frac{1 - (1 + J)^{-P}}{J} = \frac{VF}{PM} \quad (7)$$

onde P é o número de parcelas e PM o valor da prestação. No entanto, o isolamento algébrico de J nesta formulação é impossível. Adicionalmente, a Equação 7 apresenta uma singularidade numérica natural quando $J \rightarrow 0$. Para contornar essa limitação e

condicionar o problema aos métodos de busca de raízes, aplica-se a substituição de variáveis $x = 1 + J$ e a constante de proporcionalidade $k = \frac{VF}{PM}$. Dessa forma, o modelo é reescrito na forma polinomial:

$$f(x) = kx^{P+1} - (k+1)x^P + 1 = 0 \quad (8)$$

Para o **Plano A**, os parâmetros são $P = 9$ e $PM = 26.50$, o que fornece $k_A = \frac{140}{26.5} \approx 5.2830$. A função objetivo torna-se um polinômio de grau 10. Observa-se que a Equação 8 possui uma raiz trivial em $x = 1$ (o que implicaria $J = 0$, fisicamente inconsistente para um financiamento). Assim, para forçar a convergência do algoritmo para a raiz de interesse financeiro ($J > 0 \implies x > 1$), o intervalo inicial de busca deve ser estritamente superior à unidade.

Os dados processados pelo algoritmo basearam-se no intervalo inicial $[1.05, 1.30]$, que equivale a buscar uma taxa de juros entre 5% e 30% ao período, com uma tolerância rigorosa de $\epsilon < 10^{-6}$. A Tabela 4 sumariza a evolução da convergência para o Plano A.

Tabela 4: Histórico iterativo do Método da Bissecção para determinação da taxa do Plano A ($f_1(x)$).

Iteração (k)	a_k	b_k	c_k	$f_1(c_k)$	Erro (ϵ_k)
0	1.050000	1.300000	1.175000	0.67780045	0.10638297
1	1.050000	1.175000	1.112500	-0.05891906	0.05617977
2	1.112500	1.175000	1.143750	0.19422409	0.02732240
3	1.112500	1.143750	1.128125	0.04375379	0.01385041
4	1.112500	1.128125	1.120312	-0.01301356	0.00697350
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
14	1.122235	1.122250	1.122242	-0.00003841	0.00000679
15	1.122242	1.122250	1.122246	-0.00001211	0.00000339
16	1.122246	1.122250	1.122248	0.00000104	0.00000169
17	1.122246	1.122248	1.122247	-0.00000553	0.00000084

A partir da análise dos dados computacionais, verifica-se que logo na primeira iteração ($k = 1$), a detecção de alteração de sinal ($f(a_1) \cdot f(c_1) < 0$) confinou a raiz no subintervalo $[1.1125, 1.1750]$, descartando a parcela superior do domínio. Por mais que a função polinomial possua alto grau — o que frequentemente induz gradientes acentuados e compromete a estabilidade de métodos preditivos como Newton-Raphson —, o Método da Bissecção não demonstrou qualquer sensibilidade à inclinação da curva, atestando sua inquestionável robustez.

Durante as iterações subsequentes, nota-se o declínio assintótico rigoroso do erro. Na 17ª iteração, o critério de parada foi satisfeito, uma vez que o erro relativo computado $\epsilon_{17} \approx 8.49 \times 10^{-7}$ ultrapassou o limiar de tolerância imposto. O valor con-

vergado de $c_{17} \approx 1.122247$ permite extrair diretamente a taxa de juros do Plano A:

$$J_A = x_A - 1 \approx 0.122247 \quad \text{ou} \quad 12.22\% \text{ ao período.} \quad (9)$$

Observa-se ainda que a avaliação do resíduo em c_{17} retornou $f_1(1.122247) \approx -5.53 \times 10^{-6}$, comprovando numericamente que o ponto determinado é, de fato, a raiz do polinômio de financiamento.

Evolução do Erro e do Resíduo para o Plano A

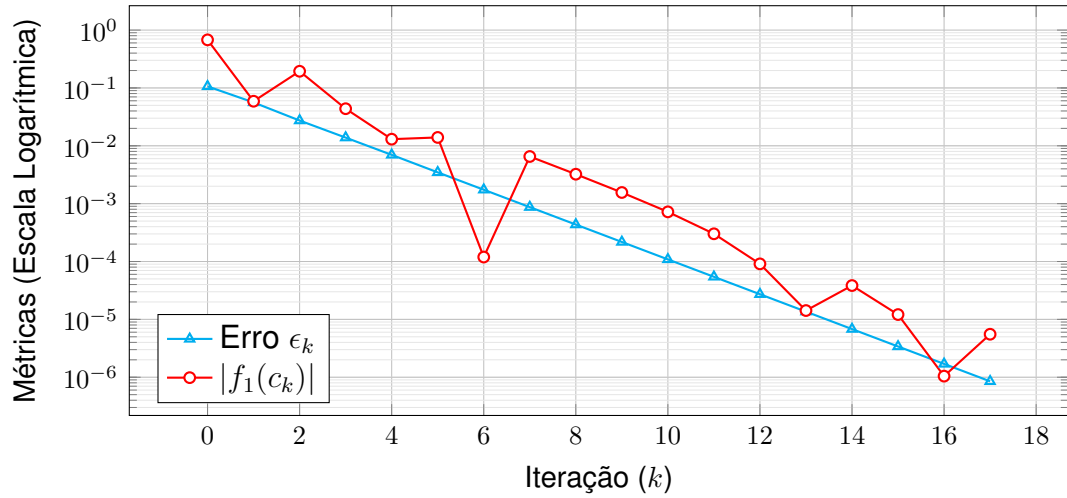


Figura 3: Gráfico de convergência atestando a contração linear do hipervolume de erro para a determinação da taxa de juros no Plano A.

Torna-se evidente que, ao replicar o mesmo arcabouço computacional para o **Plano B** ($P = 12$, $k_B \approx 6.5116$), o método revelará a taxa J_B . A tomada de decisão final pelo consumidor deverá ser pautada na comparação quantitativa das soluções extraídas, optando-se pelo plano cuja raiz computada $(x - 1)$ possua a menor magnitude. A formulação numérica provou-se altamente aderente ao problema proposto, mitigando as singularidades do modelo matemático original por meio da estratégia de substituição polinomial acoplada ao isolamento conservador da Bissecção.

2.4 Dificuldades Enfrentadas

Durante o ciclo de desenvolvimento, duas barreiras fundamentais requisitaram análise e readequação de código:

A primeira dificuldade decorreu de uma limitação algorítmica associada à manipulação em linguagem Python envolvendo o cálculo de erro relativo. O equacionamento inicial do erro demandava a divisão pela soma dos limites do intervalo $(a_k + b_k)$.

Em problemas cujo domínio submetido contemplasse raízes simétricas (como, por exemplo, funções ímpares centradas na origem onde $a_k = -b_k$), o denominador se tornava nulo, causando o encerramento abrupto da aplicação mediante a exceção `ZeroDivisionError`. A transição adaptativa do erro relativo para o erro absoluto nesses pontos críticos exigiu criteriosa instrumentação de *if-statements*, solucionando a limitação do ambiente sem desrespeitar o rigor numérico.

A segunda dificuldade envolveu a aderência aos padrões de projeto de software estruturado. A intenção de isolar a lógica matemática da escrita de arquivos resultou na necessidade de encapsular o estado iterativo. Em iterações profundas, manter um vetor histórico de dicionários em memória, para apenas no final transferi-los para a rotina de escrita CSV (camada de controle), demonstrou inicialmente um ligeiro sobrepeso na alocação de variáveis. A superação exigiu refinamento na forma como o Python instancia e mapeia essas estruturas temporárias. Ainda que houvesse maior complexidade inicial no planejamento desta abstração, as hipóteses projetuais confirmaram-se válidas, viabilizando o uso de dados de saída padronizados independentemente do método matemático em execução.

3 Método 2

3.1 Estratégia de Implementação

Descreva as escolhas que teve que fazer para poder superar eventuais dificuldades com limitações da linguagem, do ambiente de programação, etc. De que forma resolveu tratar os erros, critérios de parada, formato de saída e quaisquer outros fatores importantes para a solução dos problemas.

3.2 Estrutura dos Arquivos de Entrada/Saída

Descreva o formato dos arquivos de entrada e saída e justifique as escolhas.

3.3 Problema teste 1, 2, 3...

Use problemas das seções “Problemas Aplicados a Projetos” (ao final de cada capítulo) do livro Cálculo Numérico, de Neide Franco, para demonstrar que a sua implementação funciona e traz resultados da forma mais eficiente que lhe foi possível. Use, ao menos 3 problemas. Mostre os resultados e teste se eles realmente são a solução. Se não funcionar e você não entender o porquê, explique o que tentou e faça hipóteses.

3.4 Dificuldades enfrentadas

(Descreva as dificuldades aqui)

4 Considerações Finais

(Escreva suas considerações finais aqui)

Referências

- [1] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- [2] Robert Johansson. *Numerical Python: Scientific Computing and Data Science Applications with Numpy, SciPy and Matplotlib*. Apress, 2 edition, 2018.
- [3] Travis E Oliphant. Python for scientific computing. *Computing in Science & Engineering*, 9(3):10–20, 2007.
- [4] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020.